Map Analysis - Endpoints and SQL Server Tables

Android app **pk.gop.pulse.katchiAbadi** and backend **LandSurveyApp / API** (ASP.NET Core, SQL Server). Covers the Map Analysis screen: loading parcels, Accept on red (unmapped) and yellow (no survey) parcels, and Decline. Rows marked in amber are new in this change.

Map colours: yellow = Cane without survey (both 'Unmapped cane cells' and 'Cane without a survey': satellite shows cane but there is no survey - the surveyor surveys it; split / merge allowed), red = Surveyed but not cane (a survey exists but satellite shows no cane - the surveyor removes the survey; no split / merge), green = Done (surveyed, or survey removal recorded), purple = Declined (no further action). The colour comes from the server's **action** field for uploaded work and from the device's own tables for work not uploaded yet.

1. Flows at a glance

Flow	What the user does	Saved on device (SQLite)	Uploaded by	Endpoint	SQL Server tables
Load map	Opens Map Analysis (or downloads a mauza)	`analytics_parcels` (offline copy)	Automatic when online	`GET api/SugarcaneIntersection/GetAnalyticsParcels`	Reads `SurveyAnalyticsParcels` JOIN `Parcels` (plus the active sub-parcels of a split parcel); action from `ParcelAnalyticsAccepts` / `ParcelDeclines`
Remove survey - red (Surveyed but not cane)	REMOVE SURVEY, confirm, optional remarks; GPS saved	`parcel_accepts`, `parcel_activity_log`	**Manually**: Home > Pending Accepts	`POST api/MobileData/AddParcelAccept`	Inserts `ParcelAnalyticsAccepts`; deletes the parcel's rows from `NewSurveyParcels` / `NewSurveyPersons` / `NewSurveyPictures` (DB triggers copy them to the __DeleteLog tables); updates `Parcels` (SurveyStatusCode = 1, AttachedSurveyId = NULL; logged to Parcels_UpdateLog). GetAnalyticsParcels then reports the parcel as Accepted (drawn green)
Survey - yellow (Cane without survey)	SURVEY opens the survey form; user saves the survey (split / merge first if needed)	`active_parcels`, `new_surveys`, `survey_persons`, `survey_images`, `new_survey_lessors`, `temp_survey_log`	Manually: Saved records > Upload	`POST api/MobileData/AddSurveyData`	Writes `NewSurveyParcels`, `NewSurveyPersons`, `NewSurveyPictures`, `SowingUser_details`; inserts/updates `Parcels` (SurveyStatusCode = 2)
Decline - red or yellow	Decline, 3 pictures + remarks	`parcel_declines`	Manually: Home > Pending Declines	`POST api/MobileData/AddParcelDecline`	Inserts `ParcelDeclines`, `ParcelDeclinePictures`; JPEGs written to disk

No automatic upload. Accepts, declines and surveys are all saved on the device first and go to the server only when the surveyor taps Upload. Only the map data itself is fetched automatically when online.

2. Endpoints

Endpoint	Auth	Called from (app)	Request	Response	SQL Server effect
`GET api/SugarcaneIntersection/GetAnalyticsParcels?mauzaId=&selectedYear=`	None (no [Authorize] on controller)	`MapAnalysisRepository.loadAnalyticsParcels` - Map Analysis screen, and Menu mauza download (result cached in `analytics_parcels`)	mauzaId (int); selectedYear optional - app sends null (all years)	{ mauzaId, featureCount, features[] }; one feature per active parcel, or per active sub-parcel when the parcel was split: id, parcelId, mauzaId, parcelNo, khewat, status, grower*, cropType, parcelAcres, caneAcres, canePct, missingAcres, lat, lng, computedAcres, geometryWKT, action (Accepted / Declined / null), parentParcelId, subParcelNo, areaAssigned, surveyStatusCode	**Read only.** `SurveyAnalyticsParcels sap INNER JOIN Parcels p ON p.Id = sap.ParcelId AND p.IsActivate = 1 AND p.GeomWKT IS NOT NULL WHERE sap.MauzaId = @MauzaId`, plus an Action column (Accepted / Declined) derived from `ParcelAnalyticsAccepts` and `ParcelDeclines`
`POST api/MobileData/AddParcelAccept` **NEW**	Bearer JWT ([Authorize]); user from CNIC claim	`MapAnalysisRepository.uploadPendingAccepts` - Home > Pending Accepts card	JSON array: AcceptGuid, MauzaId, MauzaName, AreaName, ParcelId, ParcelNo, ParcelStatus, KhewatInfo, Remarks, Latitude, Longitude, DeviceCreatedDate	{ Message, Saved, Skipped, Results[] { AcceptGuid, Status, SurveysRemoved, PersonsRemoved, PicturesRemoved } } - Status is Saved, Duplicate or ParcelNotFound	One transaction per call. For each row: skip if AcceptGuid exists (Duplicate); skip if parcel missing or inactive (ParcelNotFound); else INSERT `ParcelAnalyticsAccepts` (keeps previous status), then **unsurvey**: DELETE `NewSurveyParcels / NewSurveyPersons / NewSurveyPictures WHERE ParcelId = @id` (the DB's DELETE triggers write the old rows to the __DeleteLog tables) and UPDATE `Parcels SET SurveyStatusCode = 1, AttachedSurveyId = NULL` (UPDATE trigger writes Parcels_UpdateLog). Does **not** write SurveyAnalyticsParcels (its Status column has CHECK constraint CK_SurveyAnalyticsParcels_Status). Reads `Users`.
`POST api/MobileData/AddParcelDecline`	Bearer JWT ([Authorize])	`DeclineRepositoryImpl.uploadDecline` - Home > Pending Declines > Upload all (one decline per call)	JSON array: DeclineGuid, MauzaId/Name, AreaName, ParcelId, ParcelNo, SubParcelNo, ParcelStatus, KhewatInfo, Remarks, GeomWKT, Centriod, CalculatedArea, Latitude, Longitude, Bearing, Timestamp, DeviceCreatedDate, Pic1-Pic3 (base64)	{ Message, Saved, Skipped }	Skips duplicate DeclineGuid or missing pictures. INSERT `ParcelDeclines` then `ParcelDeclinePictures` (file paths). JPEGs saved under config `uploadDir:tempUploadPathSurvey\declines\yyyy-MM-dd`. Does **not** change Parcels or SurveyAnalyticsParcels.
`POST api/MobileData/AddSurveyData`	Bearer JWT ([Authorize])	`NewSurveyRepositoryImpl.uploadSurvey` - Saved records screen (yellow-parcel survey; ParcelOperation Same / Split / Merge / New)	JSON array of ParcelDto (survey fields, Persons, Pictures, SowingPersons, GeomWKT, ParcelOperation)	{ Message, ParcelsSaved, OriginalParcelsDeactivated, ParcelsGeomUpdated }	Uses `sp_getapplock`. Writes `NewSurveyParcels`, `NewSurveyPersons`, `NewSurveyPictures`, `SowingUser_details`; INSERT/UPDATE `Parcels` (SurveyStatusCode = 2, split/merge deactivation, Geom from GeomWKT). Reads `Users`, `LandHierarchy`. Raw JSON also written to `uploadDir:tempUploadPathSurveyLog`.

3. SQL Server tables

Note: the red/yellow status comes from `SurveyAnalyticsParcels`, which is rebuilt by a job outside this repository. The API does not change that table; instead GetAnalyticsParcels reports Action = Accepted / Declined from `ParcelAnalyticsAccepts` / `ParcelDeclines`, and the app draws those parcels green / purple, so a re-run of the job cannot turn them red again. On the device, a parcel accepted, declined or surveyed is coloured the same way even before it is uploaded.

Table	Created / filled by	Key columns	Used by
`dbo.SurveyAnalyticsParcels`	**External job / script - not in the API repo.** No entity or DbSet; only queried with raw SQL.	Id, ParcelId (= Parcels.Id), MauzaId, ParcelNo, Khewat, **Status**, GrowerName, GrowerCode, GrowerMobile, CropType, ParcelAcres, CaneAcres, CanePct, MissingAcres, Lat, Lng, Year. Status values used by the app: 'Unmapped cane cells' (red), 'Cane without a survey' (yellow). Status has CHECK constraint CK_SurveyAnalyticsParcels_Status, so the API never writes it.	GetAnalyticsParcels (read only)
`dbo.Parcels`	Existing core table (entity Parcel)	Id (int), MauzaId, ParcelNo, GeomWKT / Geom, IsActivate, **SurveyStatusCode** (1 = not surveyed, 2 = surveyed, 3 = sent for revisit, 5 = revisit received), AttachedSurveyId, DeactivatedByUserId, DeactivatedAt, DeactivateReason	GetAnalyticsParcels (join), AddParcelAccept (status 1), AddSurveyData (status 2, insert/update)
`dbo.ParcelAnalyticsAccepts` **NEW**	Script `API/DbScripts/CREATE_ParcelAnalyticsAccepts.sql` (must be run once); entity ParcelAnalyticsAccept	Id (bigint identity), AcceptGuid (unique), MauzaId, MauzaName, AreaName, ParcelId, ParcelNo, ParcelStatus, KhewatInfo, Remarks, Latitude, Longitude, PreviousSurveyStatusCode, CreatedByUserId, CreatedDate, DeviceCreatedDate. Indexes on AcceptGuid (unique), ParcelId, MauzaId.	AddParcelAccept
`dbo.ParcelDeclines`	Existing (entity ParcelDecline)	Id, DeclineGuid, MauzaId, MauzaName, AreaName, ParcelId, ParcelNo, SubParcelNo, ParcelStatus, KhewatInfo, Remarks, GeomWKT, Centriod, CalculatedArea, Latitude, Longitude, CreatedByUserId, CreatedDate, DeviceCreatedDate	AddParcelDecline
`dbo.ParcelDeclinePictures`	Existing (entity ParcelDeclinePicture)	DeclineId, DeclineGuid, Pic1-Pic3 (file paths on server disk), Latitude, Longitude, Bearing, CapturedDateTime, CreatedByUserId, CreatedDate	AddParcelDecline

`dbo.NewSurveyParcels`
`dbo.NewSurveyPersons`
`dbo.NewSurveyPictures`
	Existing (entities NewSurveyParcel / Person / Picture; EF default table names)	Survey record per parcel, its owners/growers, and its pictures (files on disk, paths in table)	AddSurveyData (insert); AddParcelAccept (delete by ParcelId)

`dbo.NewSurveyParcels__DeleteLog`
`dbo.NewSurveyPersons__DeleteLog`
`dbo.NewSurveyPictures__DeleteLog`
`dbo.Parcels_UpdateLog`
	Existing audit tables filled by the database's own DELETE / UPDATE triggers (not by the API)	Copy of each deleted survey row / the Parcels row before an update	Filled when AddParcelAccept unsurveys a parcel; read them to see what an accept removed

`dbo.SowingUser_details`	Existing (entity SowingUserDetail)	Sowing persons for a surveyed parcel	AddSurveyData
`dbo.Users`	Existing (entity AppUser)	Id, CNIC, Deactivate. The JWT Name claim carries the CNIC.	All three POST endpoints (who is uploading)
`dbo.SugarcaneIntersection`	Shapefile upload (UploadSugarcane)	Satellite cane-detection cells: Mouza_Id, Area_Acre, Year, Geom, IsActive	Not read by Map Analysis directly; presumably the input of the job that fills SurveyAnalyticsParcels

4. Device tables (Room / SQLite, database kachi_abadi_db, version 32)

Table	Purpose	Key columns	Cleared / updated when
`analytics_parcels` **NEW**	Offline copy of GetAnalyticsParcels per mauza, so Map Analysis works without internet	pk, serverRowId, parcelId, mauzaId (indexed), parcelNo, khewat, status, parcelAcres, computedAcres, geometryWKT, cachedAt, action, parentParcelId, subParcelNo, areaAssigned, surveyStatusCode	Replaced for the mauza on every successful online load and on mauza download
`parcel_accepts` **NEW**	Accepts of red parcels waiting for upload	id, acceptGuid, mauzaId, mauzaName, areaName, serverParcelId (indexed), parcelNo, parcelStatus, khewatInfo, remarks, latitude, longitude, createdAt, isUploaded, lastError	isUploaded = 1 after the server answers Saved or Duplicate; lastError holds the latest failure
`parcel_declines`	Declines waiting for upload (3 picture paths)	declineGuid, serverParcelId, parcelStatus, remarks, pic1Path-pic3Path, isUploaded	isUploaded = 1 after upload; row and pictures kept
`active_parcels`	Surveyor's working parcels; a yellow Accept inserts the parcel here before the survey form opens	id (server parcel id), parcelNo, mauzaId, geomWKT, surveyStatusCode, surveyId, isActivate	surveyStatusCode = 2 when the survey is saved
`new_surveys`, `survey_persons`, `survey_images`, `new_survey_lessors`, `temp_survey_log`	Saved survey and its people, pictures and log	-	Marked uploaded after AddSurveyData succeeds
`parcel_activity_log`	Audit trail of parcel actions	action, status, details, parcelId, parcelNo, mauzaId, areaName, userId, lat, lng, createdAt	New actions: ACCEPTED, ACCEPT_UPLOADED, ACCEPT_UPLOAD_FAILED (status FAILED with the error)

5. Multi-select, split, merge and sub-parcels

Action	How	What is written	Upload
Remove survey of red parcels	Tap REMOVE SURVEY (or SELECT several + REMOVE SURVEY), one remarks dialog	One `parcel_accepts` row per parcel (own GUID, the parcel's own area), ACCEPTED in `parcel_activity_log`; parcels turn green	Pending Accepts sends them in ONE AddParcelAccept call; server unsurveys each (deletes NewSurvey* rows, SurveyStatusCode = 1)
Survey several yellow parcels / sub-parcels	SELECT, tap, SURVEY - one form	SurveyActivity batch mode: one `new_surveys` row per parcel with its own parcelId / subParcelNo / areaName, persons and images copied per survey, all in one Room transaction	Saved records uploads each survey separately (AddSurveyData, ParcelOperation Same)
Decline several parcels	SELECT, tap, DECLINE - one set of 3 pictures + remarks	One `parcel_declines` row per parcel sharing the 3 picture files (files are only deleted when no row references them), DECLINED logged per parcel	Pending Declines uploads each row (AddParcelDecline)
Split a parcel	Card SPLIT (or SELECT one + SPLIT), tap 2 points across it, APPLY SPLIT	Same contract as the main map: parent row in `active_parcels` goes inactive, 2 children with negative ids, subParcelNo 1 / 2 and the parent's area / mauza / khewat; SPLIT logged. A red parent also gets an accept record (remarks 'Split into 2 parts').	Surveying a child uploads all siblings as New with the parent id; the server creates the sub-parcels with the parent's AreaAssigned and deactivates the parent (reason Split)
Merge parcels	SELECT 2+ (same area, not red), MERGE	Opens the survey form with ParcelOperation Merge: base = first parcel, members in parcelOperationValueHi; MERGE_SELECTED logged	AddSurveyData Merge (existing pipeline)
Sub-parcels on the map	Automatic	Server: GetAnalyticsParcels returns the active sub-parcels of a split parcel (parent inactive) with the parent's analytics status. Device: local children from active_parcels replace a parent split on this device. Labels show 527/1.	-

Area safety: every row Map Analysis writes to `active_parcels` carries the parcel's own server `AreaAssigned` (now returned by GetAnalyticsParcels), never the selected-area preference, and the survey form receives areaName / mauzaId extras for the same reason. Split children copy the parent's area; the server preserves it when the parent id is sent; merges are refused across areas. A parcel therefore stays in the area it had before.

6. Deployment checklist

- Run `API/DbScripts/CREATE_ParcelAnalyticsAccepts.sql` on the SQL Server database.
- Deploy the API with the new `AddParcelAccept` action, `ParcelAnalyticsAccept` entity and DbSet.
- No change is needed to `SurveyAnalyticsParcels` or its CHECK constraint.
- DataContext now declares the audit triggers on `Parcels` / `NewSurveyParcels` / `NewSurveyPersons` / `NewSurveyPictures` (HasTrigger), so EF saves use trigger-safe SQL. Without it, a table with a trigger makes AddSurveyData fail with HTTP 500 and no survey is recorded.
- Upload order is enforced by the app: a parcel's pending accept (unsurvey) is sent before its survey, so the clean-up can never remove the new survey.
- Release the app: database migrations 29 to 32 add `analytics_parcels` (with action / sub-parcel / area columns) and `parcel_accepts` without touching existing data.